

POS para Pupuserías

Especificación técnica integral y línea base del producto

DOCUMENTO DE REFERENCIA

VERSION 1.0

Producto oficial: POS para Pupuserías

Repositorio interno: Boomwalos-POS

Stack: PHP 8.3 | Laravel 13.8 | Filament 5.7 | PHPUnit

Alcance inicial: El Salvador

Documento preparado para definir el alcance, la arquitectura, los riesgos, las pruebas y el roadmap de la primera tarea del proyecto.

1. Resumen ejecutivo

POS para Pupuserías es un sistema de punto de venta orientado a negocios gastronómicos de alta rotación. Su diferenciador no es únicamente registrar productos y pagos, sino modelar la operación real de una pupusería:

- Pedidos que se amplían mediante tandas sin reiniciar la cuenta.
- Combos configurables por slots y sabores.
- Cocina organizada por tandas y estados de preparación.
- Resumen de producción de plancha sin perder la trazabilidad de cada cliente o destino.
- Caja con apertura, cobro y arqueo ciego.
- Integración con una API fiscal externa desacoplada del POS.
- Portal QR para solicitar un documento fiscal, pendiente de implementación.
- Arquitectura preparada para varias empresas y sucursales.

La base actual es una aplicación Laravel modular basada en Service Layer, contratos y adaptadores. No se debe describir todavía como DDD completo ni como sistema listo para producción: existen componentes operativos implementados, pero faltan pruebas de integración, hardware, portal público, certificación fiscal y validación real multi-BD.

Dictamen ejecutivo

El proyecto tiene una base técnica suficiente para continuar el desarrollo de forma ordenada. Antes de venderlo formalmente deben resolverse especialmente:

1. El modo fiscal directo contra el modo Smart QR diferido.
2. La persistencia de un documento fiscal activo y su historial.
3. La resolución de tenant en todos los canales.
4. El enrutamiento de impresoras por sucursal.
5. El portal QR, la impresión QR y la integración productiva con la API fiscal.
6. El agregador de plancha con resumen y comandas individuales.

2. Identidad del producto y límites de marca

El sistema debe manejar tres niveles diferentes:

Nivel	Propósito	Ejemplo
Producto	Nombre del software vendido	POS para Pupuserías
Empresa	Cliente que contrata el sistema	Pupusería El Sabor
Sucursal	Local operativo de una empresa	Sucursal Centro

“Boomwalos-POS” puede permanecer temporalmente como nombre interno del repositorio, pero no debe aparecer como marca visible en:

- Login.
- Panel administrativo.
- POS.
- Cocina.
- Tickets.
- Portal QR.
- Documentos comerciales entregados al cliente.

La marca visual debe ser configurable por empresa mediante nombre comercial, logo, favicon, color principal, encabezado de ticket, pie de ticket y datos de contacto. Los datos fiscales deben permanecer separados de la marca visual.

3. Alcance del producto

3.1 Incluido en la primera línea de producto

- Catálogo de productos.
- Combos por slots.
- Mesas y pedidos para llevar.
- Tandas incrementales.
- Pagos en efectivo y otros métodos configurados.
- Apertura y cierre de caja.
- Arqueo ciego.
- KDS por tandas.
- Estados de cocina.
- Impresión en cola.
- Auditoría de operaciones.
- Marca configurable por empresa.
- Sucursales y asignación de personal.
- Conexión con API fiscal externa mediante adaptador.
- Base de arquitectura multiempresa.

3.2 Pendiente antes de comercialización formal

- Portal QR público.
- Generación y persistencia de tokens QR.
- Emisión CF/CCF desde el portal.
- Flujo fiscal Smart QR completo.
- Contrato productivo con proveedor fiscal.
- Webhooks productivos con resolución de tenant.
- Driver físico ESC/POS.
- QR nativo en impresoras térmicas.
- Enrutamiento de impresoras por sucursal.
- Agregador de plancha.
- Pruebas multi-BD con dos empresas reales.
- Pruebas de hardware.
- CI y ejecución completa de PHPUnit.

3.3 Fuera del alcance inicial

- Dominios personalizados por cliente.
- Expansión tributaria inmediata a Centroamérica.
- Aprovisionamiento completamente automático de tenants.

- Facturación global de suscripciones.
- Monitoreo central avanzado.
- Control completo de inventario, recetas e insumos.
- Soporte 24/7 como compromiso comercial.

Estas capacidades pueden incorporarse en fases posteriores, pero no deben anunciarse como terminadas.

4. Stack y estilo arquitectónico

4.1 Stack verificado

- PHP 8.3.
- Laravel 13.8.
- Filament 5.7.
- Spatie Permission 8.3.
- PHPUnit 12.5 como framework de pruebas configurado.
- Base de datos configurable; la producción multiempresa se orienta a una base por tenant.

4.2 Descripción arquitectónica correcta

La descripción recomendada es:

Aplicación modular basada en Service Layer, contratos, adaptadores, contexto de tenant y separación de responsabilidades.

No se debe afirmar todavía que el sistema es DDD completo. La aplicación utiliza modelos Eloquent y servicios de aplicación; la evolución futura puede introducir value objects, eventos de dominio y bounded contexts cuando exista una necesidad concreta.

4.3 Capas principales

```

Interfaz
  Filament, Livewire, portal público y endpoints HTTP
  |
Aplicación
  Casos de uso, servicios de cobro, pedidos, fiscalización, cocina e impresión
  |
Contratos
  FiscalGateway, KitchenDispatcher, CustomerTicketDispatcher,
  AuditLogger, TenantConnectionResolver y EstablishmentContext
  |
Adaptadores
  API fiscal HTTP, mock fiscal, cola de impresión, futura ESC/POS,
  futura mensajería y futuros proveedores externos
  |
Persistencia
  Base central de plataforma y base operativa independiente por empresa
  
```

5. Arquitectura SaaS multiempresa

5.1 Modelo elegido

El modelo objetivo es:

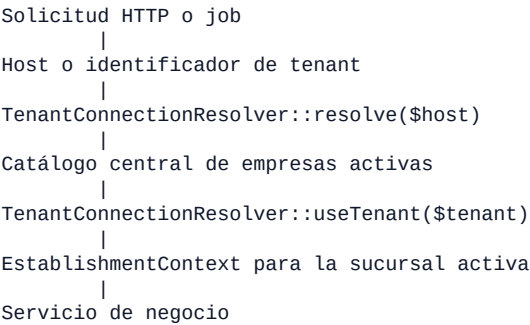
```

Base central de plataforma
  Empresas, estado, plan, slug y conexión

Base operativa por empresa
  Usuarios, sucursales, productos, pedidos, pagos, caja,
  auditoría, fiscalización, impresión y configuración
  
```

No se debe agregar `tenant_id` a todas las tablas operativas si cada empresa utiliza una base independiente. La conexión seleccionada representa el tenant.

5.2 Flujo de resolución



La configuración local conserva `TENANT_DATABASE_MODE=single` como modo de desarrollo y pruebas. La validación real requiere `TENANT_DATABASE_MODE=database` con dos empresas y dos conexiones distintas.

5.3 Canales que deben validar el tenant

Canal	Estado actual	Validación requerida
Panel administrativo	Base implementada	Probar middleware y contexto en Filament
API	Parcial	Aplicar resolución explícita por host o credencial
Portal QR	Pendiente	Resolver subdominio antes de consultar token
Webhooks	Diseño definido	Identificar tenant, validar firma y cambiar conexión
Jobs fiscales	Parcial	El job fiscal serializa <code>tenantSlug</code> ; probar todos los demás jobs
Comandos de consola	Implementado para migraciones	Probar iteración y reset de conexiones
Dos bases simultáneas	Pendiente	Prueba de aislamiento y concurrencia

5.4 Administradores y permisos

Administrador de plataforma

Gestiona la plataforma central:

- Empresas.
- Estado de cuentas.
- Planes.
- Conexiones.
- Soporte.
- Futuro aprovisionamiento.

Administrador de empresa

Gestiona solamente su empresa:

- Sucursales.
- Usuarios.
- Asignaciones.
- Productos.
- Combos.
- Mesas.

- Impresoras.
- Configuración fiscal.
- Marca.

Personal operativo

- Cajero: caja y operaciones autorizadas en sucursales asignadas.
- Cocina: KDS y tandas de su sucursal.
- Entrega: consulta y entrega de tandas autorizadas.

Toda autorización debe comprobar empresa, sucursal y permiso. No basta con ocultar una opción en la interfaz.

6. Flujo operativo del POS

6.1 Flujo de pedido

```
Seleccionar servicio
|
Seleccionar mesa o para llevar
|
Agregar producto o combo
|
Configurar slots del combo
|
Agregar notas de cocina
|
Guardar pedido
|
Crear tanda inicial
|
Enviar comanda a cocina
|
Agregar tandas posteriores sin reiniciar la cuenta
```

6.2 Tandas

Una tanda representa una entrega incremental del mismo pedido. Cada tanda conserva:

- pedido_id.
- numero_tanda.
- Estado de cocina.
- Detalles asociados.
- Mesa o destino del pedido.
- Composición individual.
- Trabajos de impresión relacionados.

Estados operativos:

```
PENDIENTE -> EN_PREPARACION -> LISTA -> ENTREGADA
```

Las transiciones deben ejecutarse mediante KitchenService y no directamente desde la interfaz.

6.3 Combos

La estructura seleccion_combo representa la composición unitaria de un combo.

Ejemplo:

```
1 combo:
  4 revueltas
  3 queso
  3 chicharrón

2 combos:
```

8 revueltas
6 queso
6 chicharrón

Debe existir una prueba que evite multiplicar dos veces la selección interna.

7. KDS y plancha

7.1 Principio central

El resumen de plancha indica cuánto producir. Las comandas individuales indican para quién separar.

El resumen nunca reemplaza, elimina ni fusiona las tandas originales.

7.2 Doble vista

Resumen de producción	Comandas de entrega
Revueltas: 42	T-001 - Mesa 5
Queso: 28	4 revueltas + 2 queso
Frijol con queso: 16	Estado: PENDIENTE
Ayote: 8	
	T-002 - Mesa 2
	6 revueltas + 1 frijol
	Estado: EN_PREPARACION

7.3 Flujo de plancha

```

PlanchaAggregationService calcula el resumen
|
Planchero prepara la cantidad total por sabor
|
Cocina conserva las tarjetas por tanda
|
Se separa y empaca por número de tanda
|
Se marca cada TandaPedido como LISTA
|
El resumen se recalcula y descuenta las tandas listas

```

No debe existir una acción de “marcar toda la plancha como lista”.

7.4 Reglas del agregador

- Filtrar por tenant y establecimiento activo.
- Incluir solamente tandas PENDIENTE y EN_PREPARACION.
- Excluir líneas canceladas.
- Multiplicar `detalle.cantidad * item.cantidad` para combos unitarios.
- Incluir únicamente productos que requieran plancha.
- Agrupar por una clave estable de producto o preparación, no solamente por nombre.
- Mantener la composición original de cada tanda.
- Reutilizar `KitchenService` para cambios de estado.
- No permitir cambios de estado desde el resumen agregado.

7.5 Contrato recomendado

```
interface PlanchaAggregationServiceInterface
{
    /**
     * @return Collection<int, PlanchaAggregateLine>
     */
    public function resumenActual(): Collection;

    /**
     * @return Collection<int, TandaPedido>
     */
    public function tandasActivas(): Collection;
}
```

El servicio debería recibir `EstablishmentContextInterface` por constructor. Si conserva un parámetro de establecimiento, debe comprobar que coincide con el contexto activo.

7.6 Riesgos operativos de plancha

- Un mismo nombre de producto puede representar preparaciones distintas.
- Una tanda puede contener productos de varias estaciones.
- Una cancelación después de iniciar preparación debe quedar auditada.
- Una modificación de pedido debe generar una nueva tanda o un evento claro, no alterar silenciosamente una tanda ya impresa.
- La separación física requiere número de tanda visible en comanda, pantalla o etiqueta.

8. Caja, pagos y auditoría

8.1 Caja

El flujo esperado es:

```
Abrir sesión con saldo inicial
|
Registrar cobros
|
Controlar métodos de pago y cambio
|
Cerrar sesión
|
Solicitar arqueo ciego
|
Comparar esperado contra contado
|
Registrar diferencia y auditoría
```

8.2 Idempotencia de pagos

El cobro debe protegerse contra:

- Doble clic.
- Reintento de red.
- Dos cajeros cobrando el mismo pedido.
- Reprocesamiento de una petición.

La protección debe combinar estado de pedido, transacción, bloqueo y restricciones únicas donde correspondan.

8.3 Auditoría

El sistema registra eventos de operación mediante `AuditLogger` y `EventoAuditoria`.

Estado real:

Registro append-only por convención de aplicación; todavía no es inmutable a nivel de motor SQL.

Debe reforzarse con:

- Política de no actualización.
- Política de no eliminación.
- Permisos restringidos.
- Retención y respaldo.
- Pruebas de intento de modificación.
- Posible trigger o tabla histórica protegida.

9. Arquitectura fiscal

9.1 Frontera del sistema

```

POS para Pupuserías
|
| HMAC, timeout, idempotencia y outbox
v
API fiscal externa
|
| Certificado .p12, firma JWS, transmisión y respuesta
v
Ministerio de Hacienda

```

El POS no debe manejar directamente la certificación .p12 si la API fiscal externa es la frontera elegida.

9.2 Modos fiscales requeridos

Debe existir una política por empresa o configuración fiscal:

Emisión directa

Pago -> registrar venta -> enviar DTE -> webhook -> documento emitido

Solicitud Smart QR

```

Pago -> registrar venta local -> generar token QR
Cliente solicita CF o CCF
-> crear solicitud fiscal
-> enviar DTE a API externa
-> recibir webhook
-> actualizar documento
-> habilitar descarga

```

9.3 Estado actual y discrepancia importante

Actualmente, cuando la fiscalización está habilitada, `FiscalSaleRegistrar` registra la venta después del pago y `FiscalOutboxService` la envía a la API externa.

`FiscalDocumentoService` requiere que la venta ya esté sincronizada y crea o actualiza el documento local; no representa todavía el caso de uso público ni dispara por sí solo una nueva emisión desde el portal.

Por lo tanto, Smart QR es todavía una funcionalidad de diseño. Antes de implementarla debe definirse si la API externa:

1. Pre-registra ventas sin emitir DTE.
2. Emite el DTE en el primer envío.
3. Tiene un endpoint separado para solicitar el documento después.

La interfaz fiscal debe reflejar esa decisión. Una interfaz genérica `enviarVenta()` puede no ser suficiente para distinguir registrar, emitir, consultar e invalidar.

9.4 Estados fiscales

El código actual contempla principalmente:

Venta fiscal: NO, SINCRONIZADO, ENVIO_FALLIDO
Documento: PENDIENTE, EMITIDO, RECHAZADO

Para invalidaciones y reemplazos debe añadirse una estrategia explícita:

- Estado INVALIDADO y REEMPLAZADO, o
- Tabla de historial/eventos fiscales separada.

9.5 Un documento activo por pedido

Regla de negocio:

Un pedido solo puede tener un documento fiscal activo en estado PENDIENTE o EMITIDO. Puede conservar historial de rechazados, invalidados y reemplazados.

La base actual utiliza `UNIQUE(pedido_id, tipo_documento)`, lo cual permitiría CF y CCF simultáneos. Antes del portal debe hacerse una migración controlada:

1. Detectar duplicados activos.
2. Resolver los casos existentes.
3. Definir documento actual versus historial.
4. Aplicar la restricción elegida.
5. Probar dos solicitudes simultáneas CF/CCF.

No se debe depender únicamente de `lockForUpdate()` sin una estrategia de persistencia que soporte concurrencia real.

10. Portal QR público

10.1 Flujo objetivo

```

Ticket pagado
|
Crear token opaco aleatorio
|
Guardar SHA-256 del token en la BD del tenant
|
Imprimir QR con subdominio del tenant
|
Cliente escanea
|
Resolver tenant por host
|
Validar token, vigencia y estado
|
Mostrar resumen mínimo de venta
|
Elegir CF o CCF
|
Validar DTO fiscal
|
Crear solicitud idempotente
|
Transmitir a API fiscal
|
Procesar webhook
|
Mostrar descarga PDF/JSON

```

10.2 Token

El token debe ser:

- Aleatorio y de alta entropía.
- Opaco.
- No derivado de pedido_id.
- Almacenado únicamente como hash.
- Expirable.
- Reutilizable en modo lectura después de emitir.
- No regenerado durante una reimpresión.

Tabla propuesta en la base del tenant:

```
solicitudes_ticket_qr
  id
  establecimiento_id
  pedido_id
  token_hash
  estado
  expires_at
  solicitado_at
  ip_solicitante
  created_at
  updated_at
```

10.3 Estado del token

El estado mínimo recomendado es:

```
ACTIVO -> EN_PROCESO -> EMITIDO
              \-> RECHAZADO
ACTIVO -> EXPIRADO
```

Si se decide no guardar EN_PROCESO, el portal debe derivar ese estado desde DocumentoFiscal y rechazar reenvíos duplicados.

10.4 Seguridad del portal

- Rate limiting por IP y token.
- Protección contra replay.
- No exponer IDs secuenciales como credenciales.
- No mostrar datos fiscales innecesarios.
- Política Referrer-Policy: no-referrer.
- No registrar el token crudo en logs.
- Respuestas idempotentes.
- Expiración configurable.
- Auditoría de solicitudes.

10.5 URL

Desarrollo:

```
http://{tenant_slug}.pos.localhost/factura/{token}
```

Producción propuesta:

```
https://{tenant_slug}.pospupuserias.sv/factura/{token}
```

El dominio debe ser configurable. El subdominio debe resolver el tenant antes de consultar el token.

11. Impresión y ESC/POS

11.1 Estado actual

Actualmente existe una capa de cola que crea registros de TrabajoImpresion. Todavía no existe un driver físico completo para USB, TCP o Bluetooth.

11.2 Riesgo actual de sucursal

La tabla de impresoras actual no tiene una relación con establecimiento_id, y los servicios de cola buscan la primera impresora por tipo.

Esto puede enviar una comanda de una sucursal a la impresora de otra sucursal de la misma empresa.

Antes de implementar hardware se requiere:

```
impresoras
  establecimiento_id
  nombre
  tipo
  driver
  configuracion
  habilitada
  capacidades
```

La consulta debe filtrar por:

```
tenant actual + establecimiento activo + tipo de trabajo
```

11.3 Driver

La arquitectura debe permitir:

```
EscPosNetworkDriver
EscPosUsbDriver
EscPosBluetoothDriver
MockPrinterDriver
```

Todos deben implementar el mismo contrato y soportar:

- Impresión de texto.
- Corte.
- Reintentos.
- Estado del trabajo.
- QR nativo.
- Fallback textual.
- Registro de errores.

11.4 QR térmico

El driver debe probarse con hardware homologado para:

- Comando QR nativo.
- Tamaño de módulo.
- Corrección de error.
- Codificación.
- Legibilidad.
- Reimpresión del mismo token.
- Fallback a URL completa.

12. SOLID y contratos

Single Responsibility

Separar responsabilidades de:

- Cobro.
- Validación de pagos.
- Registro fiscal.
- Cocina.
- Impresión.
- Auditoría.
- Contexto de sucursal.

Open/Closed

Agregar un proveedor fiscal, impresora o canal de cocina sin modificar el núcleo de cobro.

Liskov Substitution

HttpFiscalGateway y MockFiscalGateway deben cumplir el mismo contrato y comportamiento observable.

Interface Segregation

Contratos pequeños:

- FiscalGatewayInterface.
- KitchenDispatcherInterface.
- CustomerTicketDispatcherInterface.
- AuditLoggerInterface.
- TenantConnectionResolverInterface.
- EstablishmentContextInterface.
- BrandingServiceInterface.

Dependency Inversion

Los servicios principales deben recibir interfaces por constructor. No deben depender directamente de drivers HTTP, impresoras concretas o resolutores de tenant específicos.

13. Seguridad y privacidad

Aislamiento

- Nunca aceptar un tenant_id del cliente como autoridad.
- Resolver tenant desde host, credencial o catálogo central verificado.
- Validar sucursal mediante contexto y permisos.
- Evitar modelos centrales usando accidentalmente la conexión tenant.
- Limpiar el contexto al finalizar cada request o job.

Fiscal

- HMAC con timestamp.
- Anti-replay.

- Idempotencia por clave de reintento.
- Validación de payload.
- No guardar certificados .p12 en el POS si la API externa es responsable de ellos.

Operativa

- Permisos por rol y sucursal.
- Auditoría de cambios críticos.
- Protección de caja.
- Logs sin datos fiscales innecesarios.
- Backups por tenant.
- Pruebas de restauración.

14. Estado actual verificable

Módulo	Estado real
POS	Implementado en código; circuito de hardware pendiente
Pedidos, combos y tandas	Implementado; concurrencia y carga pendientes
KDS	Implementado por tandas; prueba física pendiente
Resumen de plancha	Diseño cerrado; código pendiente
Caja y arqueo	Implementado; piloto operativo pendiente
Auditoría	Implementada a nivel aplicación; blindaje SQL pendiente
API fiscal	Adaptador HMAC y mock; proveedor real pendiente
Smart QR	Diseño cerrado; código pendiente
Impresión	Cola implementada; driver físico pendiente
QR ESC/POS	Pendiente
Contexto tenant	Base implementada; cobertura por canal pendiente
Multi-BD	Arquitectura parcial; prueba con dos tenants pendiente
Plataforma central	Registro y panel base; aprovisionamiento, billing y monitoreo futuros
PHPUnit	Suite configurada; ejecución local bloqueada por dependencias del entorno

Verificaciones de entorno conocidas

La ejecución completa de PHPUnit requiere resolver dependencias del entorno local, incluyendo ext-intl y ext-zip. Hasta ejecutarla en CI o un entorno preparado, ningún módulo debe marcarse como validado por pruebas automatizadas.

15. Planes comerciales de referencia

Los precios son estimados y deben validarse con costos reales.

Plan	Enfoque	Alcance de referencia
Básico	Operación	POS, mesas, tandas, combos, KDS, caja y tickets internos
Smart QR	Fiscalización bajo solicitud	Básico más QR, portal CF/CCF y cuota DTE base
Pro	Emisión directa	Smart QR más emisión directa, cuota ampliada e invalidaciones cuando estén listas

Plan	Enfoque	Alcance de referencia
Cadena	Enterprise	Multi-sucursal, multi-caja, reportes, insumos y soporte prioritario cuando estén validados

Costos a considerar:

- DTE por documento.
- Infraestructura y bases separadas.
- Almacenamiento de PDF/JSON.
- Mensajería.
- Impresoras y tablets.
- Soporte.
- Respaldos.
- Monitoreo.

La política fiscal y el modo de emisión deben ser aprobados por el cliente y su asesor tributario. El sistema no debe venderse como herramienta para ocultar ventas o eludir obligaciones.

16. Pruebas obligatorias

16.1 POS y pedidos

- Crear pedido para mesa.
- Crear pedido para llevar.
- Agregar segunda tanda.
- Validar combo exacto.
- Rechazar combo incompleto.
- Evitar doble cobro.
- Reintentar una operación sin duplicar pago.

16.2 Cocina y plancha

- Mostrar dos tandas del mismo sabor.
- Sumar combo por cantidad.
- Excluir línea cancelada.
- Descontar tanda al marcarla lista.
- Mantener comandas separadas.
- No permitir marcar el resumen completo como listo.
- Filtrar por estación de cocina.
- Evitar mezclar sucursales.

16.3 Fiscal

- Emisión directa en caja.
- Solicitud Smart QR.
- Solicitudes CF y CCF simultáneas.
- Una sola solicitud fiscal activa por pedido.

- Rechazo y reintento.
- Invalidación y reemplazo.
- Webhook duplicado.
- Webhook fuera de orden.
- Webhook con firma inválida.
- Webhook de otro tenant.
- Descarga de PDF/JSON.

16.4 Multiempresa

- Empresa A no ve Empresa B.
- Sucursal A no ve Sucursal B.
- Cajero no opera sucursal no asignada.
- Job conserva tenant correcto.
- Webhook resuelve tenant correcto.
- Impresora no recibe trabajos de otra sucursal.
- Dos bases pueden operar simultáneamente.

16.5 Hardware

- POS en pantalla táctil.
- Impresora de red.
- QR escaneable.
- Reimpresión con mismo token.
- Desconexión y reintento.
- Impresión de contingencia.

17. Roadmap priorizado

P0 - Decisiones y bloqueos

1. Confirmar modo fiscal por empresa: directo, Smart QR o ambos configurables.
2. Definir contrato real de la API fiscal externa.
3. Diseñar documento actual e historial fiscal.
4. Definir estado del token QR durante procesamiento.
5. Corregir impresoras por sucursal.

P1 - Validación de arquitectura

6. Resolver dependencias y ejecutar PHPUnit en CI.
7. Probar dos tenants con dos bases de datos.
8. Probar panel, API, jobs y webhooks con contexto correcto.
9. Añadir pruebas de aislamiento y permisos.

P2 - Funcionalidades comerciales

10. Construir portal QR público.
11. Implementar caso de uso público fiscal.

12. Implementar drivers ESC/POS y QR nativo.
13. Implementar PlanchaAggregationService y doble vista.
14. Implementar estados y reintentos productivos de documentos.

P3 - Piloto y negocio

15. Probar con hardware real.
16. Ejecutar piloto en una pupusería.
17. Medir tiempos de pedido, cocina, impresión y emisión.
18. Calcular costos reales.
19. Cerrar precios y contratos comerciales.

18. Criterios de aceptación de la tarea 1

La tarea 1 puede considerarse completa cuando:

- El documento técnico está aprobado.
- El producto se identifica como POS para Pupuserías.
- Boomwalos queda solo como identificador interno.
- El alcance inicial se limita a El Salvador.
- Se diferencia claramente código implementado de producción validada.
- Se documenta la arquitectura de tenant y sucursal.
- Se documentan los modos fiscales directo y Smart QR.
- Se documenta que Smart QR todavía requiere implementación.
- La plancha tiene resumen agregado y comandas individuales.
- Se documenta la unicidad fiscal y su historial.
- Se documenta el riesgo de impresoras por sucursal.
- Se establecen pruebas obligatorias.
- Se establece un roadmap con prioridades.
- El documento se puede adjuntar a la herramienta de gestión del proyecto.

19. Decisiones pendientes para aprobación técnica

Antes de desarrollar las tareas P1 y P2, el líder técnico debe aprobar:

1. Si Smart QR pre-registra la venta o transmite el DTE solamente después de la solicitud.
2. Si el documento actual e historial se separan en tablas.
3. Cómo se enruta el webhook por tenant.
4. Cómo se asigna una impresora a una sucursal y estación.
5. Qué productos pertenecen a la estación PLANCHA.
6. Qué estado muestra el portal mientras la API fiscal procesa.
7. Qué proveedor fiscal y contrato se utilizarán en producción.
8. Qué hardware se homologará para el piloto.

20. Referencias del repositorio

Aplicación

- app/Services/PedidoService.php
- app/Services/CobroService.php
- app/Services/KitchenService.php
- app/Services/FiscalDocumentoService.php
- app/Services/FiscalWebhookService.php
- app/Services/FiscalSaleRegistrar.php
- app/Application/Fiscal/FiscalClient.php
- app/Application/Fiscal/FiscalOutboxService.php
- app/Application/Printing/QueueCustomerTicket.php
- app/Application/Kitchen/QueueKitchenBatch.php
- app/Filament/Pages/Kitchen/KitchenDisplay.php
- app/Models/TandaPedido.php
- app/Models/Impresora.php
- app/Jobs/EnviarVentasFiscalesJob.php

Multiempresa

- config/tenancy.php
- app/Context/TenantContext.php
- app/Context/EstablishmentContext.php
- app/Http/Middleware/ResolveTenant.php
- app/Services/Platform/TenantConnectionResolver.php
- app/Providers/Filament/PlatformPanelProvider.php
- app/Providers/Filament/AdminPanelProvider.php
- database/migrations/2026_08_15_000003_create_platform_tenants_table.php
- database/migrations/2026_08_15_000004_create_platform_tenant_connections_table.php
- database/migrations/2026_08_15_000005_create_establecimiento_usuario_table.php

Persistencia y fiscal

- database/migrations/2026_08_08_212932_create_documento_fiscals_table.php
- database/migrations/2026_08_15_000001_create_tablas_fiscales_table.php
- database/migrations/2026_08_08_212929_create_impresoras_table.php
- tests/Feature/TenantIsolationTest.php
- tests/Feature/SolidContractsTest.php
- tests/Feature/Fiscal/FiscalOutboxTest.php
- tests/Feature/Fiscal/FiscalDocumentoServiceTest.php

Cierre

POS para Pupuserías tiene una base operativa real y una dirección técnica coherente. La arquitectura permite crecer sin mezclar la marca del primer cliente con el producto, y permite separar empresa, sucursal, operación, fiscalización e infraestructura.

La prioridad correcta no es agregar muchas pantallas de inmediato. Es cerrar las decisiones que pueden obligar a rehacer datos o flujos: modo fiscal, historial de documentos, aislamiento por canal, impresión por sucursal y agregación de plancha.

Una vez completadas esas decisiones y sus pruebas, el sistema podrá avanzar de una base de código funcional a un producto POS comercializable y operable por varias pupuserías.